

Benchmark Requirements/Metrics

Table of Contents

Benchmark Requirements/Metrics	1
Purpose	3
Scope	3
Research Objective	3
Functional Requirements	4
Non-functional Requirements	6
Common / Base Metrics	7
Use-Case based Metrics	8
Chat / General LLM.....	8
RAG	9
Classification	10
Agentic Tool-Use/Workflows	11
Experimental Methodology Requirements	12
Controlled Variables.....	12
Benchmark Repetitions	12
Model Selection	13
Data Storage.....	13
Validation	14
Feasibility Assessment	15

Purpose

This document defines the requirements for local Large Language Model (LLM) benchmarking environment.

The purpose of the environment is to systematically evaluate open-weight LLMs across different parameter sizes and use cases, measuring their performance, resource consumption, output quality and reability.

Scope

The benchmark will be evaluated across 4 main use cases:

- Chat and General Reasoning (CGR)
- Retrieval-Augmented Generation (RAG)
- Classification
- Agentic Tool Use

The benchmarking will be done with mock data.

Research Objective

The benchmark will provide sufficient evidence to answer the main question:

“To what extent can locally hosted open-weight LLMs, running on limited hardware, support chat, retrieval, classification and agentic-tool use applications, and under what circumstances would additional hardware investment be justified.”

Functional Requirements

Functional Requirements describe what the benchmarking environment must be capable of performing,

ID	Requirement	Priority
FR-01	The system must execute inference using locally hosted LLMs.	MUST
FR-02	The system must support benchmarking multiple models and parameter sizes.	MUST
FR-03	The system must execute predefined benchmark prompts.	MUST
FR-04	The system must measure and record inference performance.	MUST
FR-05	The system must measure and record hardware resource consumption.	MUST
FR-06	The system must store benchmark results in a structured local database.	MUST
FR-07	The system must record model configuration and execution environment details.	MUST
FR-08	The system must support repeated benchmark executions.	MUST
FR-09	The system must record model responses and execution errors.	MUST
FR-10	The system must evaluate results against predefined use-case-specific criteria.	MUST
FR-11	The system should support automated execution of complete benchmark suites.	SHOULD

FR-12	The system should support exporting results to CSV or JSON.	SHOULD
FR-13	The system should support comparing models across performance and quality metrics.	SHOULD
FR-14	The system could provide a visualization dashboard for benchmark results.	COULD
FR-15	The system could support configurable benchmark workloads and concurrency tests.	COULD

Non-functional Requirements

ID	Requirement	Priority
NFR-01	Model inference must execute on the allocated ASUS hardware.	Must
NFR-02	The benchmark must operate without external AI inference APIs.	Must
NFR-03	Benchmark execution must be possible without internet access after initial setup.	Must
NFR-04	Benchmark datasets, model outputs, and measurements must remain stored locally during execution.	Must
NFR-05	The environment must use consistent and documented configurations for comparable experiments.	Must
NFR-06	Benchmark executions must be reproducible using recorded configurations, software versions, and datasets.	Must
NFR-07	Failed, timed-out, and crashed executions must be recorded rather than silently discarded.	Must
NFR-08	The system should use containerization to simplify deployment and configuration management.	Should
NFR-09	The system should support adding new models and benchmark categories without major architectural changes.	Should

NFR-10	The benchmark database must support subsequent analysis without requiring tests to be rerun.	Must
--------	--	------

Common / Base Metrics

ID	Metric	Unit	What it tells you
BASE-01	Model load time	s	How long the model takes to become ready
BASE-02	Time to First Token (TTFT)	s	Initial responsiveness
BASE-03	Total latency	s	Total time to complete the request
BASE-04	Input tokens	tokens	Size of the input workload
BASE-05	Output tokens	tokens	Amount of generated output
BASE-06	Prompt processing speed	tok/s	How quickly the model processes context
BASE-07	Generation speed	tok/s	How quickly it generates output
BASE-08	Average CPU usage	%	CPU demand during execution
BASE-09	Peak CPU usage	%	Maximum CPU demand
BASE-10	Average RAM usage	GB	Typical memory consumption
BASE-11	Peak RAM usage	GB	Maximum memory requirement
BASE-12	Swap usage	GB	Whether the system is exceeding practical RAM capacity
BASE-13	Model storage size	GB	Disk requirement
BASE-14	GPU utilization	%	GPU usage, if applicable

BASE-15	VRAM usage	GB	GPU memory requirement, if applicable
BASE-16	Execution success	pass/fail	Whether the request completed
BASE-17	Timeout	yes/no	Whether execution exceeded the allowed limit
BASE-18	Crash/OOM	yes/no	Whether the model caused a runtime/memory failure
BASE-19	Output format validity	%	Whether the output conforms to the required format/schema
BASE-20	Context length	tokens	How much context was supplied

Use-Case based Metrics

Chat / General LLM

ID	Metric	Unit	What it measures
CHAT-01	Answer correctness	score / %	Whether the answer is correct
CHAT-02	Instruction following	score / %	Whether the model followed the requested instructions
CHAT-03	Relevance	score / %	Whether the answer actually addresses the question
CHAT-04	Factual accuracy	score / %	Whether factual claims are correct
CHAT-05	Hallucination rate	%	Unsupported/incorrect claims
CHAT-06	Structured output compliance	%	Whether requested JSON/format is valid
CHAT-07	Long-context quality degradation	% change	Quality change as context grows

CHAT-08	Response consistency	%	Whether repeated runs produce acceptable outcomes
CHAT-09	Abstention accuracy	%	Whether the model correctly says it doesn't know when appropriate

RAG

Retrieval

ID	Metric	Unit	What it measures
RAG-01	Recall-Top-K-Results	%	Whether the relevant document appears in the top K
RAG-02	Precision-Top-K-Results	%	How many retrieved documents are actually relevant
RAG-03	MRR	score	Whether the relevant document is ranked highly
RAG-04	Retrieval latency	ms/s	Time required to retrieve context

Generation

ID	Metric	Unit	What it measures
RAG-05	Answer correctness	% / score	Whether the final answer is correct
RAG-06	Groundedness	% / score	Whether claims are supported by retrieved context
RAG-07	Answer completeness	% / score	Whether all relevant

			information was included
RAG-08	Unsupported claim rate	%	Claims not supported by retrieved information
RAG-09	Citation accuracy	%	Whether citations actually support the claims
RAG-10	Abstention accuracy	%	Whether the model avoids inventing answers when information isn't available

Classification

ID	Metric	Unit	What it measures
CLS-01	Accuracy	%	Overall correct classifications
CLS-02	Precision	%	Correct positive predictions
CLS-03	Recall	%	Correctly identified actual cases
CLS-04	Macro F1	score	Overall balanced classification quality
CLS-05	Per-class F1	score	Performance for each category
CLS-06	Confusion matrix	matrix	Which categories are confused
CLS-07	Invalid output rate	%	Outputs outside allowed classes
CLS-08	Classification consistency	%	Stability across repeated runs
CLS-09	Confidence/calibration	score	Whether confidence reflects actual correctness, if available

Agentic Tool-Use/Workflows

ID	Metric	Unit	What it measures
TOOL-01	End-to-end task success	%	Whether the complete task was correctly completed
TOOL-02	Tool selection accuracy	%	Whether the correct tool was selected
TOOL-03	Argument accuracy	%	Whether correct parameters were generated
TOOL-04	Schema compliance	%	Whether tool calls match the required schema
TOOL-05	Tool call count	count	Number of calls required
TOOL-06	Unnecessary tool calls	count	Inefficient actions
TOOL-07	Multi-step success rate	%	Ability to complete multi-step workflows
TOOL-08	Error recovery rate	%	Ability to recover from tool/API failures
TOOL-09	Task completion latency	s	Time to complete the complete workflow
TOOL-10	Incorrect/unsafe tool invocation rate	%	Attempts to invoke inappropriate tools/actions

Experimental Methodology Requirements

Controlled Variables

To ensure fair comparisons, the following variables must be documented and controlled where applicable:

- Hardware configuration and available memory.
- Operating system and virtualization configuration.
- Docker and inference runtime versions.
- Model name, version, parameter count, and quantization.
- Context window and maximum output tokens.
- Temperature, sampling parameters, and random seed where supported.
- Benchmark dataset and prompt versions.
- Concurrency level and background workloads.
- Model warm-up and caching conditions.

If settings cannot be made equivalent across models, differences must be documented.

Benchmark Repetitions

Each benchmark task should initially be executed at least three times per model configuration.

Additional repetitions may be required for tasks exhibiting high variability or unreliable outcomes.

Results must include individual execution records as well as aggregate statistics, such as mean, median, and relevant percentiles.

Model Selection

Candidate models will be selected using the following criteria:

- Availability of model weights for local execution.
- Compatibility with the selected inference runtime.
- Parameter size and expected hardware requirements.
- Suitability for the target use cases.
- Quantization availability.
- Context window support.
- License and potential commercial-use restrictions.
- Available documentation and model maintenance.

The initial evaluation will consider approximately 1–3B, 4–5B, 7–9B, and 12–14B parameter tiers, with larger models used selectively for hardware-limit investigation.

Data Storage

The benchmark database must retain:

- Unique experiment and execution identifiers.
- Model and configuration information.
- Benchmark dataset and task identifiers.
- Timestamps and execution duration.
- Raw outputs and expected outputs.
- Hardware measurements.
- Evaluation scores.
- Execution status and error information.

The database design must allow multiple runs of the same model and task without overwriting previous results.

Validation

Before full-scale benchmarking, the measurement system must be validated using a small pilot experiment.

The pilot must verify that:

1. The model receives the expected prompts.
2. Responses are correctly captured.
3. Timing and resource metrics are recorded accurately within documented measurement limitations.
4. Failed requests are stored.
5. Results can be retrieved and compared from the database.
6. The inference pipeline works without external AI services.

Feasibility Assessment

The benchmark results will support a feasibility assessment across four dimensions.

Dimension	Assessment
Technical feasibility	Whether the model can execute successfully on the allocated hardware.
Functional feasibility	Whether the model achieves acceptable results for the target use cases.
Operational feasibility	Whether performance, memory consumption, and reliability support practical deployment.
Financial feasibility	Whether potential benefits justify infrastructure and operating costs.